

Agentic Deobfuscation: An Attacker's Playbook

Binary Cartography

Tim Blazytko



@mr_phrazer



synthesis.to



tim@blazytko.to

About Tim

- independent security researcher, consultant & trainer
- PhD in binary program analysis & reverse engineering
- focus on (de)obfuscation & automation
- trainings on reverse engineering & software deobfuscation



Setting the Scene



Agentic reverse engineering



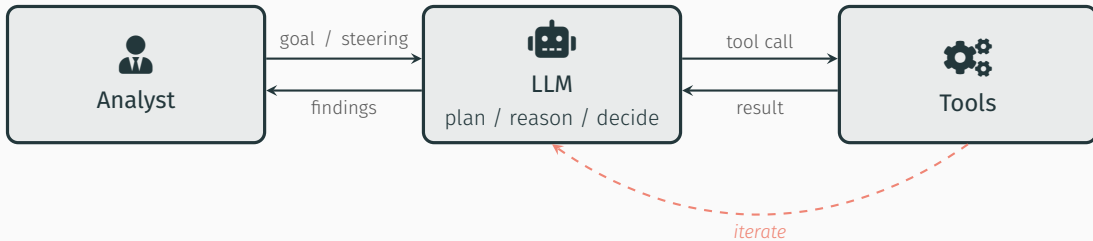
Deobfuscation playbook



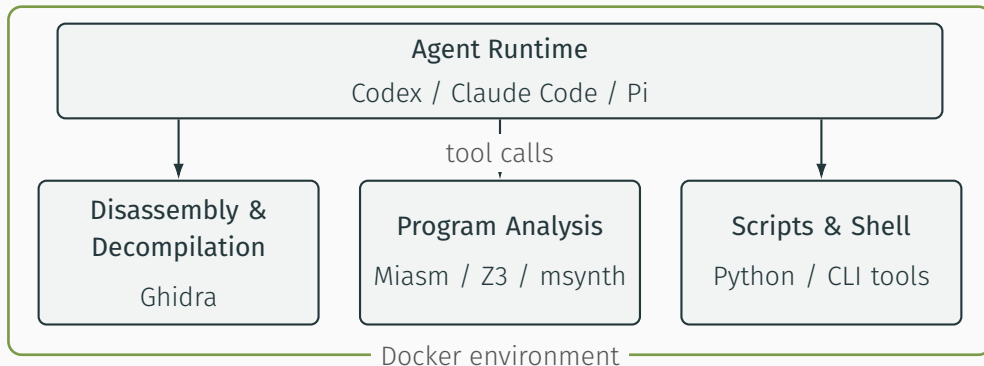
Demo

Agentic Reverse Engineering

The Agent Loop



Analysis Environment



Agentic Deobfuscation Playbook

- **analysis scope**
 - static only, or execution, instrumentation, debugging, traces?
 - provide tools, runtime access, and data within that scope
- **model capability — rule of thumb**
 - frontier models / more reasoning: allow more autonomy
 - smaller models: smaller tasks and more explicit guidance

Attacker's Playbook

- set the goal
- guide the analysis
- verify each step
- resolve blockers
- keep a project record
- build reusable skills

Set the Goal

- define the goal precisely: scope, output, and success criteria
- give context; let the agent draft the goal, then review it
- keep the goal active with the harness's **/goal** command
- finish when the agreed checks pass; report blockers or limits

Example

- **goal** minimize the selected function's CFG
- **verification** emulate both versions from identical states on 1,000 random inputs
- **accept only if** the CFG is smaller; outputs and memory effects match in every test

- provide existing findings, scripts, and relevant examples
- let the agent choose the method where it can make progress
- provide a procedure or smaller task when guidance is needed
- reconsider the procedure when its assumptions fail

Verify Each Step

choose checks for the current goal and analysis step

Examples

- **deflattening**: compare block transitions with execution traces
- **reconstruction**: compare outputs and memory effects
- **expression simplification**: prove equivalence with SMT solvers

optional: spawn an adversarial agent to challenge the findings

Resolve Blockers

- locate stalls: anti-emulation loops or other countermeasures
- check missing state, lost context, and tool limitations
- change strategies that stop producing useful results
- hard blockers often need **expert diagnosis and intervention**
- **optional:** ask a review agent for alternative approaches
- hand the analysis back once the blocker is resolved

Keep a Project Record

- use one analysis directory for notes, scripts, outputs, and tool state
- clearly document the goal, current status, and next steps
- preserve failed attempts; separate hypotheses from verified findings
- resume from the existing project after a context reset

Build Reusable Skills

- keep the working implementation and its verification checks
- ask the agent to summarize the method and build a skill
- generalize beyond the sample; state prerequisites and limits
- reuse the skill where its assumptions hold and check each result
- investigate exceptions and update the skill after validation

Demo

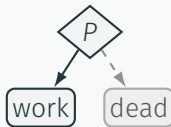
Obfuscation Primitives

Mixed Boolean-Arithmetic

$$(x \oplus y) + 2(x \wedge y) \\ \equiv \\ x + y$$

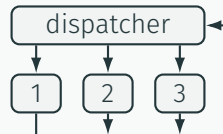
arithmetic mixed with
bitwise operations

Opaque Predicates



fixed branch outcomes

Control-flow Flattening



dispatcher-based
control flow

Removing Opaque Predicates with SE & SMT

We prove that only one branch can be taken.

```
y = x + 200  
x = x + y  
next_block = if (x != x - 1) then a else b
```

Symbolic Execution

$$c := (x + x + 200 \neq x + x + 199) ? a : b$$

SMT Solving

- $\text{SMT}(c = a) \Rightarrow \text{SAT}$ — branch can be taken
- $\text{SMT}(c = b) \Rightarrow \text{UNSAT}$ — branch can never be taken

Replace the conditional branch with a direct jump to a .

Wrap-Up

Summary

- define precise goals and provide the required analysis tools
- verify each step; use expert guidance to resolve blockers
- generalize tested deobfuscation methods into reusable skills

Webinar slides & code

<https://github.com/mrphrazer/binary-cartography>

Software Deobfuscation Techniques for Automated and Agentic Reverse Engineering

Hexacon

Paris · October 12–15, 2026

[Registration](#)

RE//VERSE

Orlando · March 14–17, 2027

[Registration](#)

Training topics

- Reverse Engineering & Binary Program Analysis
- Software Deobfuscation
- Agentic Reverse Engineering & Malware Analysis
- Android & Firmware Reverse Engineering



synthesis.to

Private trainings — on-site & remote

Tim Blazytko | tim@blazytko.to